

## Task 3 — BookStore Data Layer (LINQ and EF Core)

### User Stories

As the owner of the bookstore, I want to:

- Move all my data from temporary memory to a real database that survives application restarts.
- Have the database structure created automatically from the application code, so I never have to write SQL by hand.
- Ask any question about my data quickly and get the answer in a single trip to the database.
- Trust that the application never returns incorrect or partial results.

### Requirements

- The application must talk to SQL Server through Entity Framework Core.
- The database schema must be generated from C# code (Code First) and updated through migrations.
- The schema must enforce the same business rules from Task 1 (no negative prices, unique customer emails, deleting a category must not break existing books, and so on).
- Old purchases must keep their original price even if the book price changes later.
- All queries must be efficient. Fetching a list with related data must not produce a separate database call per record.
- Read-only queries should not waste resources tracking entities the application is not going to update.

### Tasks

1. Set up Entity Framework Core for the bookstore. Configure the DbContext, the connection string, and all entities.
2. Use Code First migrations to create the database. Make sure all relationships and constraints are enforced at the database level.
3. Seed the database with sample data on first run.
4. List all books together with their category and author in a single query.
5. Get the top 5 best-selling books.
6. Get every customer with the number of purchases they have made.
7. List categories that contain more than 5 books.
8. Get every book that costs more than the average book price.
9. Get customers who have never made a purchase.

10. Show the total revenue grouped by month.
11. Search books by a keyword in the title (case-insensitive).
12. Implement pagination for the books list (page number and page size as inputs).
13. Add a new book, update its price, and delete a book using EF Core.
14. Find any query in your code that has the N+1 problem and fix it.
15. Make read-only queries efficient by skipping change tracking where it is not needed.

#### Submission Rules

- A working .NET 10 console or class library project in a public GitHub repo.
- README must explain how to run the migrations and the application.
- Include the generated migration files in the repo.